LAB 7: FUZZING FOR INPUT VALIDATION BUGS

Software Testing Laboratory — Hypothesis Fuzz Testing
Student: G.V.N.SREEGARSHA | SRN: PES1UG24CS163
Repository: harsha2240/L6_L7_35_customs_duty | Folder: lab7

Objective

Identify input-validation failures in the customs duty Python application using Hypothesis, implement defensive validation, and verify that malformed inputs do not cause unhandled exceptions.

Scope and method

The existing functions parse_declaration(), duty_amount(), and hs_chapter() were tested with Hypothesis-generated text and None inputs. The initial buggy version was tested before making fixes. The application was then updated to return None for invalid inputs and retested.

1. Original application — successful execution

The original application produced the expected results for valid sample inputs.

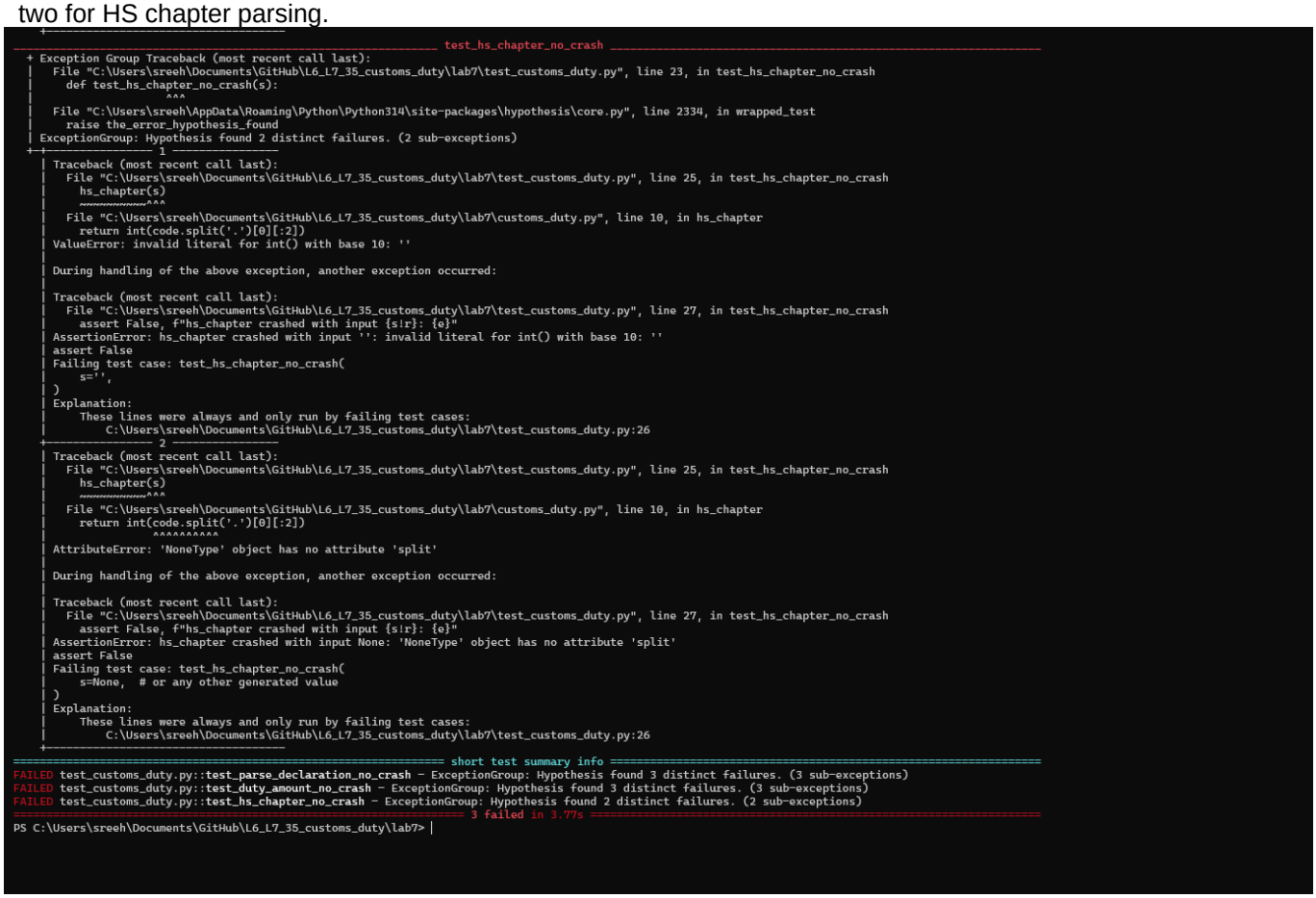
```
PS C:\Users\sreeh\Documents\GitHub\L6_L7_35_customs_duty\lab7> py customs_duty.py
==== Running parse_declaration ====
Enter category:value (e.g. electronics:25000): electronics:25000
Category and Value: ('electronics', 25000.0)

==== Running duty_amount ====
Enter value@rate (e.g. 25000@20): 25000@20
Duty Amount: 5000.0

==== Running hs_chapter ====
Enter an HS code (e.g. 8517.12): 8517.12
Chapter: 85
PS C:\Users\sreeh\Documents\GitHub\L6_L7_35_customs_duty\lab7>
```

2. Hypothesis tests — failures before fixing

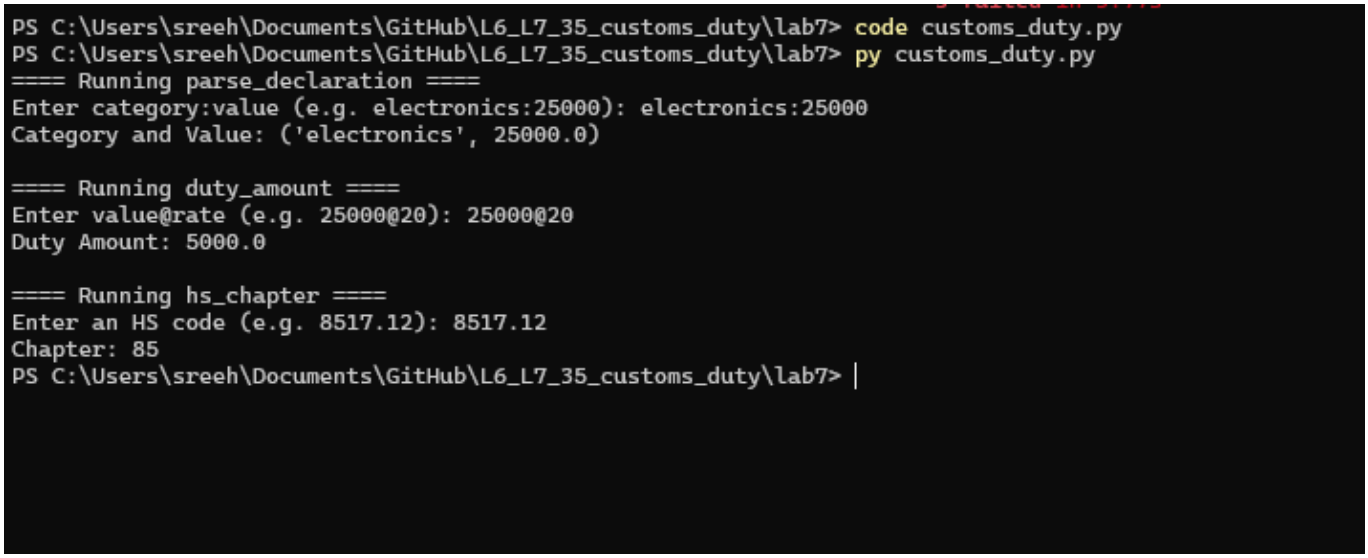
All three property-based tests failed on the original application, demonstrating unhandled exceptions for malformed input. Hypothesis reported three distinct failures for declaration parsing, three for duty calculation, and two for HS chapter parsing.



Observed outcome: 3 failed in 3.77s.

3. Fixed application — successful execution

After adding input validation, the application continued to produce the correct outputs for the same valid inputs: declaration (electronics, 25000.0), duty 5000.0, and chapter 85.

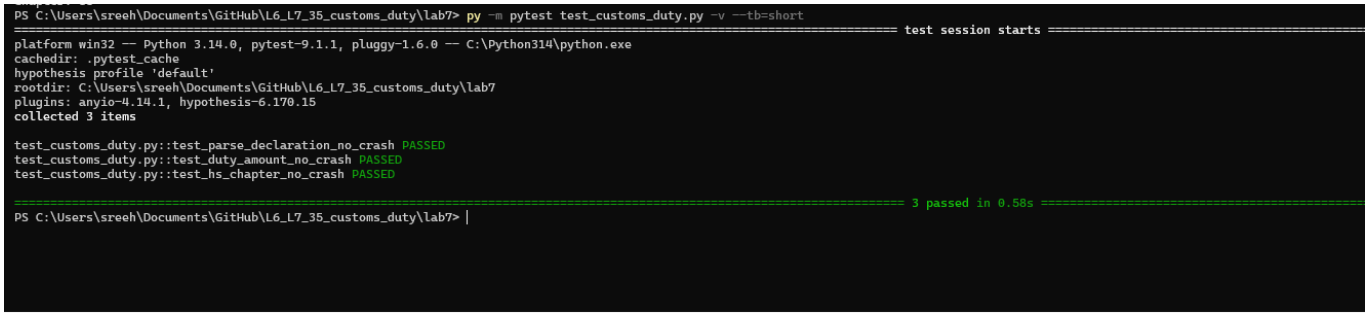


Validation changes

The revised functions reject malformed separators, invalid types, non-finite or negative amounts, and invalid HS-code formats. Invalid inputs return None rather than raising unhandled exceptions. The duty-rate range of 0–100 and the accepted HS-code patterns were chosen for this exercise.

4. Hypothesis tests — passing after fixing

The same three fuzz tests were rerun against the updated application. All three passed.



Test summary

Test	Result
test_parse_declaration_no_crash	PASSED
test_duty_amount_no_crash	PASSED
test_hs_chapter_no_crash	PASSED

Final result: 3 passed in 0.58s.

Conclusion

Hypothesis exposed input-validation failures in all three original functions. After defensive validation was added, the application retained its expected behavior for valid inputs and passed the three no-crash fuzz tests. These results demonstrate robustness against the tested malformed inputs, not exhaustive correctness for every possible input.